

CSU Computer and Information Sciences

Factory Simulator Sandbox Game

Kyle Addy – Fall, 2019

Project Advisor – Dr. Hayes

Table of Contents

Section 1: Statement of purpose.....	3
Section 2: Research/Background	4
Section 3: Hardware/Software required for project.....	5
Section 4: Final project screen shots/ explanation	6
Section 5: Test Plan	9
Section 6: Test Results.....	16
Section 7: Challenges Overcome.....	21
Section 8: Future Enhancements	22
Section 9 Final Defense Presentation Slides	24

Section 1: Statement of purpose

My project is a video game where the only way to interact with the environment is through robots that the player builds and programs to perform different tasks. This game is enjoyable and helps teach the basic knowledge of any programming language such as If Else Statements, For Loops, and While Loops. It does this by using “programming blocks” that the user organizes in any order, which each robot will then use as a list of commands to follow.

The main mechanic I am using to teach the basics of programming is with “programming blocks”. This is how I started to learn the basics was by solving problems by using drag and drop programming blocks in a software called ¹Alice. But instead of setting up a scene and giving commands for the objects in the scene like in Alice, the programming blocks in this game will be put in a list and will act as a list of commands that each individual robot follows. For an example in the game you will want to have an almost constant supply of materials. In order to do this in a time efficient manor, the player will need to have more than a single robot gathering materials. The player will have to set up a robot to gather materials and put them into a storage chest near the mine, but since there is not just a button that the player would push that tells the robot to do this the player will need to use the programming blocks that they have access to in order to solve this problem. Some of these blocks would be an If statement to check to see if the storage chest is full, if it's not full then it will then proceed to turn around to the mine and mine some material. Once it mines the materials then it will turn back around and deposit the materials into storage. Lastly, the player will need to have all of these programming blocks inside of a while Loop block so that the robot continuously mines material. The player then will need to have another robot pick up the materials from the storage and bring them to where they are needed. A problem that will occur though is that the mine and production areas may be far apart and because the robots have a limited length of programming blocks, the player will need to solve the issue of the robots not being able to move the full length. They will be able to do this by having multiple robots to pass the material off to like a relay. Another way to solve this if they player doesn't want to use material to build more robots for this would be to program a single robot to move forward while there is not chest in front and do this X number of times by using a for loop or by using a While loop. This is only one example of a problem that the player will need to solve, there are many more such as programming robots to patrol your base to detect for enemies.

¹ <https://www.alice.org/>

Section 2: Research/Background

The largest amount of information that I had to learn was how ²Unity works and how to use it to make a game. Unity is the software, called a game engine, that I used to create my video game. I started learning unity in December of 2018 and since then have learned an incredible amount of information, not only about the engine itself but about the C# programming language, which is what Unity uses.

I first started on YouTube by watching tutorials on how to start your game development process. I came across many great sources of information such as the YouTube channels ³Jimmy Vegas and ⁴Brakeys. Both of these channels have detailed tutorials on multiple aspects of Unity. I then found at that Brakeys has a ⁵Discord Channel that now has over twenty-five thousand members. Discord is an application that lets people easily meet up and communicate with each other online. This discord server has many channels where you are able to go on and ask any question you want, from questions about Art to C# to Unity. So, whenever I was not able to find the answer I needed on Google or YouTube, I was able to go onto this server and ask someone for help. While getting help on this server. I ended up meeting one of my best sources of information and one of my good friends, Garry Stanfill.

Garry helped me out with a problem I was having that involved game objects that were moving but were not being rendered in the correct location. I Then found out that Garry was the owner of an Indie Game Studio called Devil Kitty Games. I Ended up joining Garry and his team and started making more games and learning everything Garry knew about Unity. One of the best things that I learned from Garry was the use of Game Events. Game Events allow me to connect aspects of one part of my game to other aspects but without a reference. Not having to reference another object in Unity means that I am able to Develop and test different parts of my game a lot quicker than before. Its like getting your first cell phone when all you have had was a tethered house phone.

² <https://unity.com/>

³ https://www.youtube.com/channel/UCRMXHQ2rJ9_0CHS7mhL7erg

⁴ <https://www.youtube.com/user/Brackeys>

⁵ <https://discordapp.com/>

Section 3: Hardware/Software required for project

Software Used

- Game Maker Studio
 - Used to make a prototype of the game. Used this software because at the time this is what I was using and had no experience with Unity yet.
- Unity
 - Used to make the final version of the game. Used Unity because of its countless sources of information and tutorials that can be found online. Unity also uses C# to program applications while Game Maker Studio used its own custom more simplified Programming Language that is not able to be used in other applications. Unity also has free assets that developers can use to help develop games. One such asset is ProBuilder which allows the ability to create 3D models within Unity.
- Visual Studio
 - Used this application as my text editor for my C# scripts. Visual studio has integrations with Unity that allows me to easily use Unity's API.
- ClickUp
 - Used this online website / application to keep track of tasks and to set milestones for development. This application is the best that I have found at helping manage my projects, Other applications being Hack-N-Plan and Trello.

Hardware Used

- A PC running the windows 10 operating system
 - This was the hardware used because this is all I had access to develop on.

Section 4: Final project screen shots/ explanation

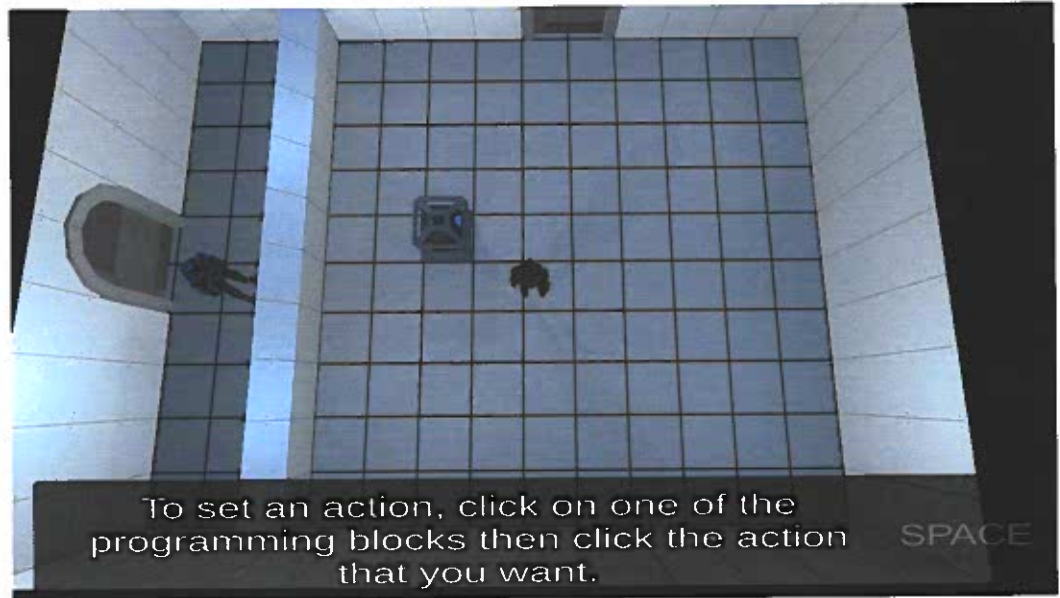
Full of Bugs Main Menu. Here The player is able to Load a previously saved game, start a new game, play the tutorial or quit the game.



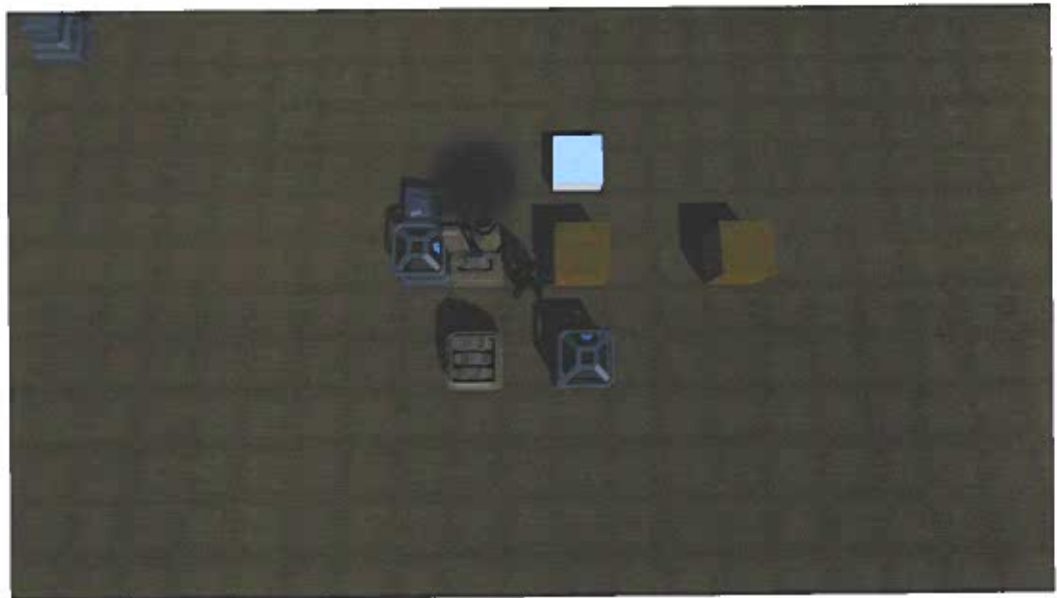
Level Generation Menu. In this menu the player is able to change the settings that are used when generating a new level such as size of world and the spawn chance of each ore.



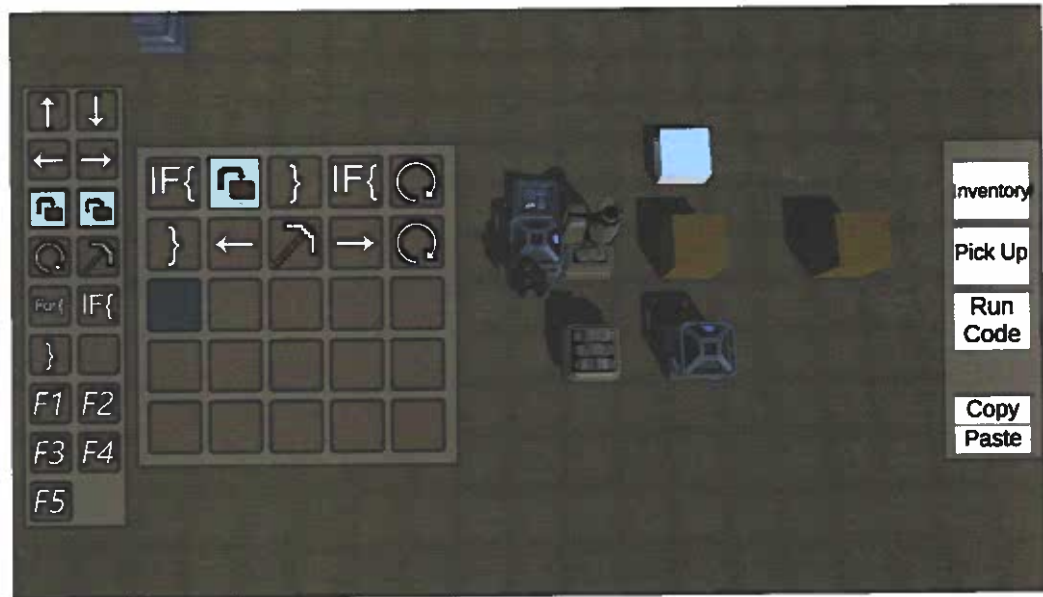
Tutorial Level. In the tutorial the player can learn the controls and mechanics of the game in a safe space.



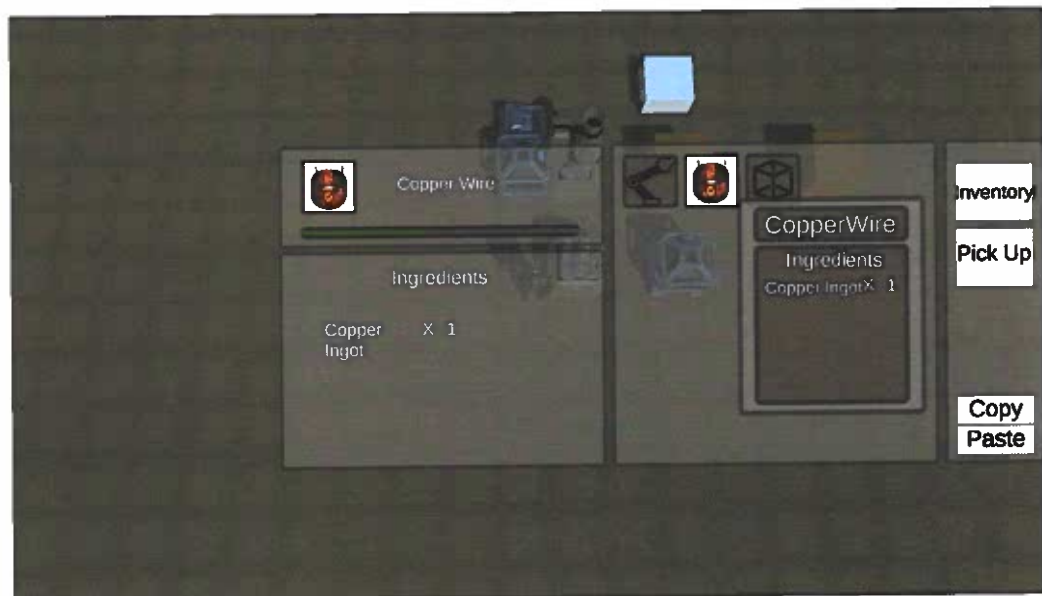
In this image the player is in the middle of a small factory that is mining and turning iron ore into iron ingots



In this image the player is programming the robot to mine ore and deposit it into a refinery to be converted to an ingot



In this image the player is viewing the recipe of the Copper Wire while the assembler is in the middle of crafting a copper wire from a copper ingot



Section 5: Test Plan

Meeting Application Requirements:

The application needs to pass all requirements that are in the Senior Project Requirements Specification Document. These Requirements shall be checked by a developer and given pass or fail depending on how it did.

Playtesting:

Having Multiple people playtest through the game is essential to see how others react and overcome challenges in the game. For playtesting multiple people shall play through the tutorial and start a new game after being given the playtesting introduction specified in the Senior Project Testing Script Document.

All Application Requirements:

Requirement Number/Type	1 Functional
Description	Ability to move player with WASD keys
Rationale	Player must be able to move around the environment
Fit Criteria	During Game play the player character must move when the WASD keys are pressed.
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	2 Functional
Description	Player character must not collide with other objects
Rationale	Player must not intersect with other objects
Fit Criteria	During Game play the player character must stop or slide along the edge of another object rather than intersect with it.
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	3 Functional
Description	Can open player inventory GUI
Rationale	Player must be able to access the players inventory/crafting screen
Fit Criteria	During Game play the player GUI must open when the inventory button is pressed
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	4 Functional
Description	Can open Storage GUI
Rationale	Player must be able to access the GUI of the Storage Box Object
Fit Criteria	During Game play the Storage Box GUI must open when the Storage Box object is clicked on
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	5 Functional
Description	Can open Assembler GUI
Rationale	Player must be able to access the Assembler GUI screen
Fit Criteria	During Game play the Assembler GUI must open when the Assembler object is clicked on
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	6 Functional
Description	Can open Refinery GUI
Rationale	Player must be able to access the Refinery GUI screen
Fit Criteria	During Game play the Refinery GUI must open when the Refinery object is clicked on
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	7 Functional
Description	Can open Robot GUI
Rationale	Player must be able to access the Robot GUI screen
Fit Criteria	During Game play the Robot GUI must open when the Robot object is clicked on
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	8 Functional
Description	Can place all placeable objects into the world
Rationale	This is the only way for these game objects to be used
Fit Criteria	During Game play the player will Ctrl + Click a placeable object from their inventory. A blueprint will be created and follows the mouse while sticking to a grid. When the player clicks again then the object will be created into the world. Placeable objects are: Robot, Assembler, refinery, Storage box
Dependence	3
History	Created: 3/18/2019

Requirement Number/Type	9 Functional
Description	Ability to Create a new game
Rationale	Player must be able to create a new game so they can play the game
Fit Criteria	In the main menu the new game button must create a new world and start it
Dependence	N/A
History	Created: 3/18/2019

Requirement Number/Type	10 Functional
Description	Ability to save the current game
Rationale	Player must be able to save a world so they can come back to that world at a later date
Fit Criteria	During game play in the pause menu the Save game button must save the current state of the world/game into a save file
Dependence	9
History	Created: 3/18/2019

Requirement Number/Type	11 Functional
Description	Ability to load a previously saved world
Rationale	Player must be able to load a previously saved game so they can continue playing that world
Fit Criteria	In the main menu the load world button must load the last saved game
Dependence	10
History	Created: 3/18/2019

Requirement Number/Type	12 Functional
Description	Ability for the player to craft an item
Rationale	Player must be able to do this to craft items without a assembler/refinery
Fit Criteria	During game play in the player Inventory GUI clicking on an item to craft must consume the needed materials from the player's inventory, add the item to the crafting queue and be placed in the players inventory when crafted
Dependence	3
History	Created: 3/18/2019

Requirement Number/Type	13 Functional
Description	Ability for the Assembler to craft an item
Rationale	Assembler is the only object that can craft some items so it must be able to craft them so the player can make those items
Fit Criteria	During game play in the Assembler GUI clicking on an item to craft must consume the needed materials from the Assembler's inventory, add the item to the crafting queue and be placed in the players inventory when crafted
Dependence	5
History	Created: 3/18/2019

Requirement Number/Type	14 Functional
Description	Ability for the Refinery to craft an item
Rationale	Refinery is the only object that can craft some items so it must be able to craft them so the player can make those items
Fit Criteria	During game play in the Refinery GUI clicking on an item to craft must consume the needed materials from the Refinery's inventory, add the item to the crafting queue and be placed in the players inventory when crafted
Dependence	6
History	Created: 3/18/2019

Requirement Number/Type	15 Functional
Description	Ability for the player to transfer items from the player's inventory to other inventories
Rationale	The ability to move items around is needed to craft other items and to store items
Fit Criteria	During game play in all of the inventory screens Shift + Clicking on an item in the player inventory will be moved to the other inventory if there is an open inventory slot
Dependence	3,4,5,6,7
History	Created: 3/18/2019

Requirement Number/Type	16 Functional
Description	Ability for the player to transfer items from the other inventories to the player's inventory
Rationale	The ability to move items around is needed to craft other items and to store items
Fit Criteria	During game play in all of the inventory screens Shift + Clicking on an item in the other inventory will be moved to the player's inventory if there is an open inventory slot
Dependence	3,4,5,6,7
History	Created: 3/18/2019

Requirement Number/Type	17 Functional
Description	Ability to program a Robot
Rationale	The ability to program the Robot to perform actions is the main mechanic for teaching people the basics of coding logic
Fit Criteria	During game play in the Robot GUI clicking on any of the coding blocks must add it to the selected programming array and must be performed by the robot
Dependence	7
History	Created: 3/18/2019

Requirement Number/Type	18 Functional
Description	Ability to program the robot with every programming block
Rationale	The ability to program the Robot to perform actions is the main mechanic for teaching people the basics of coding logic
Fit Criteria	During game play in the Robot GUI clicking each programming block will place the correct action in the programming array and the robot must perform the actions in the correct order. Programming blocks: move forward/backward, turn left/right, mining, deposit, withdraw, If statement, For loop, repeat
Dependence	7
History	Created: 3/18/2019

Requirement Number/Type	19 Functional
Description	Robot must not collide with other objects
Rationale	The robot should not be intersecting with other objects as this can cause the game to look weird
Fit Criteria	During gameplay when the robot is moving around it must not move into the player and must avoid grid spaces that are occupied
Dependence	N/A
History	Created: 3/19/2019

Requirement Number/Type	20 Functional
Description	Player must be able to get every item
Rationale	If there is an item the player is not able to get then the player might not be able to advance far in the game
Fit Criteria	During game play go through each item and check to see if can be crafted or gathered
Dependence	N/A
History	Created: 3/19/2019

Requirement Number/Type	21 Art and Style
Description	Every item needs a model and/or a 2d sprite to visualize it
Rationale	If an item does not have a model and/or a 2d sprite then the player may not know what they are looking at.
Fit Criteria	Play the test scene and view every item to see if they have a model and/or a 2d sprite
Dependence	N/A
History	Created: 4/10/2019

Requirement Number/Type	22 Art and Style
Description	Every item that has a model must have their animation working
Rationale	If an animation does not work the player will not have an easy way of seeing if a machine is working
Fit Criteria	Play the test scene and view every item that has a model and see if its animation is working
Dependence	21
History	Created: 4/10/2029

Section 6: Test Results

Application Requirements Test Results:

For the Application requirements, almost all of them passed. Only two of the twenty-two requirements failed. Those two requirements were Requirements twenty-one and twenty-two. Both of these Requirements are part of the Art section of the requirements. These are not that important but are still needed for the application to go into an alpha release.

Requirement 21: Every item needs a model and/or a 2d sprite to visualize it .

This requirement failed because some of the game objects such as the Computer and Storage Box did not have a in game model. Having a model for every object is important because the visuals of something help players identify what that object is at a glance and causes game play to flow more smoothly. Having all the models help sell the emersion of the game which helps get players into the game.

Requirement 22: Every item that has a model must have their animation working

This requirement failed because not all the models in the game had an animation to go with it. Animations are important in this application because it allows the player to see if an object in the game is working just by looking at it and removes the need for the player to interact with every single object to make sure it is working.

Neither of these requirements were fixed because they are not a “game breaking” requirement. As in the game is still able to be played even if these two requirements fail.

Play Testing Results:

I was able to have four people playtests through the application. These four people and their experience with computer games are the following:

- 1) Dylan way -Little experience with computer games
- 2) Matthew Germroth – Has experience with computer games
- 3) Taylor Perry- Has experience with computer games
- 4) Cody Coxwell- Has some experience with computer games

Although it is better to have even more play testers, any amount is better than none. Out of all the testing I found play testers to be the most valuable as the developer can become immune to problems in their game because they have been working on it for so long. Bringing in fresh eyes can help the developer see the big glaring problems that the game has that the developer might not realize is there. In my situation I found that I needed to work on the tutorial a lot more and was able to get great feedback on what type of additions I should make to the tutorial to make it more effective.

After each player completed a playing thought he game, this meant they had completed the tutorial and had played inside of a newly created level, I would ask them five questions. These five questions and the general responses to each were the following:

- 1) On a scale of 1-10, How would you rate your overall experience while playing?

Average: 7.75

For the scale of 1-10 I told the play testers that 1 would be the worst experience they have had while playing a game and 10 being the best experience. So, to get a .75 average was really nice to see. Especially since the game is not even in alpha yet. This just means that just the main mechanics of the game is a lot of fun to play around with.

- 2) On a scale of 1-10, How easy was it to pick up on the controls?

Average: 8.75

This score of 8.75 tells me that the layout of controls fits well and is not too complicated for new players to pick up on. Having controls be simple and easy to pick up on allows players to jump into the game and enjoy it faster. Though while watching my play testers I did have to remind them on some of the controls. So, this tells me that I need to improve the tutorial to make learning the controls easier, smoother and more rememberable so that players don't run into the problem of not remembering what controls did what. Another thing that I decided that I want to add as a help screen in the pause menu so that if a player does forget one of the controls then they are able to just hit pause and check it really quick.

- 3) On a scale of 1-10, How easy was it to learn how to program the robots?
What about it made it easy/hard to learn?

Average: 8.75

For the application to receive a 8.75 score on how easy it is to learn how to use the main mechanic of the game really is nice to see. As a developer you do not want you players to have a hard time learning your game, especially if what they are learning is how to utilize the main mechanic of your game. Some things that the testers told me that helped with learning the programming aspect of the game was the tutorial and how simple the programming block commands are. From this I know that I am on the right track. I also know that the tutorial is

more important than I first initially thought. I definitely need to improve the tutorial when I continue on with this game.

4) Do you have any mechanics that you would like to see added?

By asking the testers this I am able to get a sense on what path I should take when continuing this game. It also allows me to see what my target audience wants from this type of game. The following are what ideas that I received from my testers:

- **Quick Access buttons for player crafting**

I really like this idea for quick access buttons. This will allow the player to quickly craft items while having easy access to the rest of the game. By adding this the player would no longer have to open their inventory to craft items. This would also allow there to be less amount of inputs the player must do to achieve what their goal. This would also be fairly easy to accomplish as I just need to add a button to the screen, and have it call already existing functions.

- **Ability to rotate buildings when placing them**

This was actually something I was already wanting to add but never got to adding it as it is not required. This would also be easy to add to the game, it just takes me turning the rotation of an object when the player presses a button.

- **Ability to zoom in and out with the camera**

I actually tried to add this during development but decided against it as if the camera zoomed out then they would be able to see past the borders of the level, they would also be able to see enemies before the enemies can see them once I add enemies to the game. I may still add this feature in the future as it would be helpful for finding ores or other objectives or I might add a different feature that does a similar job.

- **More environments**

This suggestion is pretty easy to do as all it entails is adding more variety to the visuals of the game. This will be something that is done in the future as it increases playability and immersion for the player.

- **Player Mining**

Player mining will be something that I add to the game. At first, I was not sure because I wanted the player to use the robots to mine. But after some thought I realized that it will still be better for the player to use the robots as it would be quicker and less tedious in the long run. But adding the ability for

the player to mine without a robot would help remove some tedious aspects of waiting in the start of starting a new game.

- **Fighting Element**

Fighting is something I want to add in the game and was actually in the original scope for the game. Though I will not be adding it the way the play tester wanted it where the player can fight. Rather I will have enemies that can attack the player and in order for the player to defend they must program a robot to defend them. This will make the player feel venerable while playing and keep them from just running around the whole map with no care of danger since they could fight back themselves.

- **Building Buildings**

- o I'm not sure about adding building to the game. Though it would be easy to add as the system for building is already in the game. The reason I may not add this is because I don't think building really fits the feel of the game. I also don't have any way of adding roofs to the buildings so if I added buildings it would only be walls that the player could place.

- **Ability to go into third person view**

- o A third person camera would be easy to add to the game, but I will not be adding it. I won't be adding it because the top own view fits the game better and allows the player to focus and plan out a program or factory. If I added a third person camera, I would also have to add a lot more visuals to the game such as a sky box or more terrain pass the edges of the map since the player would be able to see that area then.

5) Do you have any comments or suggestions about any aspect of the game?

I asked this question because it allows the testers to give input on any part of the game and not just new mechanics. Almost every suggestion that the testers gave dealt with the tutorial aspect of the game, the only one that did not deal with the tutorial was to make the small level even small. This would be easy to do but I don't think I will as I don't want the level to be so small that there is no exploration or challenge in finding materials or objectives.

I really liked what was suggested for the tutorial, one person said that I should use visual aids to show the player what keys to press down and what happens when you press each button. Another suggestion was to require the player to press the key or complete an objective before the tutorial would go to the next part. Lastly, I was asked to reduce the amount of text that the tutorial showed to the player at one time. By having less text, the player is able to

focus more on what I am trying to teach them. By having so much text at a time it can overwhelm the player and may cause them to miss something important.

Section 7: Challenges Overcome

Assembler and Refinery Crafting

The Assembler and Refinery use the same system for crafting, the only difference is what they craft. Because of this I was able to write one system that crafts anything but restricted what the player is able to choose to craft depending on if they were interacting with an assembler or a refinery. I kept running into the problem of the crafting system not being modular as in if I wanted to be able to add more items that can be crafted to the game then I would have to add custom statements to the crafting system each time. I was able to improve the system to be more modular by using Scriptable Objects for my game items. Scriptable Objects are similar to a class. It acts like a template that I am able to create each game item with. Within the template I have an array of structs that keep track of what other game items and how many of each are needed to craft the selected item. The Refinery and Assembler keeps "Storage" of the recipe items by using an array of ItemSO, which is the name of the Scriptable object for the game items. The Crafting system then iterates through this array and compares the item at each index and if it matches one of needed items from the recipe then it adds the index number to a list for that found recipe item. After running through the array, it then checks if each list contains the needed number of items. If enough items are found, then it removes those items from the inventory and sets an empty slot to the crafted item.

Adding Functions to the Robot

The way the Robot works is by having an array of strings, this array is what stores the actions that the robot needs to perform. Each index in this array corresponds to a programming block in the robot. The robot then iterates through the array and performs each action specified in each array index. This becomes a problem when I wanted to add the ability for the player to program functions that can be called within the robot's program. This is because to do so requires that I keep track of multiple locations so I can return to the correct location in code. To overcome this problem, I decided to change how the robot "runs" through its code. Now the robot iterates through the programming array (array of actions) and adds each action to a list. When the robot comes to a function it then iterates through the function's array adding those actions to the list. Once the robot finishes iterating through its programming array and adds the actions to the list, it then iterates through the list to make sure the program is balanced (every if statement as an end brace) Once the code is confirmed to be balanced the robot then iterates through the list and performs the actions.

Section 8: Future Enhancements

For Future enhancements I would like to :

Go into detail, make each point a sub heading with a paragraph or two

Recode how the robots perform their given tasks.

Currently the robots see what the action they need to be performing is and call the needed functions need to perform the action. But this can cause the functions to be called multiple times if the task takes longer than one frame. I want to change this so that the robot only has to call the needed function one time and then the function takes care of the rest. This will help with testing and adding new actions that the robot can perform.

Add more models to the game.

Adding more models to the game helps make the game feel more polished and helps the player not only feel more immersed into the game but also be able to identify game objects quickly just by the way they look.

Redo the User Interfaces.

Currently the user interfaces are basic and displaying just enough info to play the game. I want to improve the menus by adding tooltips and quick access buttons to streamline the player's ability to interact with the elements in the game. I also want to add art to the menus to make them look nicer.

Add music and sounds.

Adding music and sounds will help the player feel more immersed in the game and keep the player playing for longer. Sounds will also allow me to have sound cues in the game, so the player is able to know something is happening even if they cannot directly see it.

Add more background story.

I want to add more information the player can gather while playing in order to help push the reason why the player wants to play. A backstory also will help give reason for enemies and an end goal. Currently the story of Full of Bugs is that the world became so dependent on technology and the advancement of robotics that the robots became self-aware. After becoming self-aware, they then proceeded to try and eliminate all humans left in the world. This caused humans to go into hiding. But humans don't have the needed skills to survive without technology so they must use their current skills and knowledge of robotics to try and fight back with what they can.

Implement a campaign mode.

I want the evil robots to find the hiding place of the humans, this causes the player to leave the hiding place and enter the outside world. This campaign can replace the tutorial as the first few levels of the campaign can introduce the player to the world, mechanics and controls. This campaign will add more playability to the game and make the learning process of the game to be more fun.

Add enemies.

Enemies will add danger to the world of Full of Bugs and will cause the player to not only think about automation but also about defense. Enemies also add more challenge to the game. Some enemies could be a Spider robot, a simple walking robot that walks to the player and melees them. A small drone like robot. This robot could fly over walls and crevasses and can shoot at the player. Drop Ship, this is a large ship that flies over the world and drops the other robots down to attack the player if it spots the player.

Section 9 Final Defense Presentation Slides

CSU Computer and Information Sciences

Factory Simulator Sandbox Game

Kyle Addy – Fall, 2019

Project Advisor – Dr. Hayes

Purpose

To help teach new and upcoming programmers the basic logic behind programming statements such as, “If” Statements, “For” Loops and Functions by creating a fun game.

Game Screen Shots

Main Menu



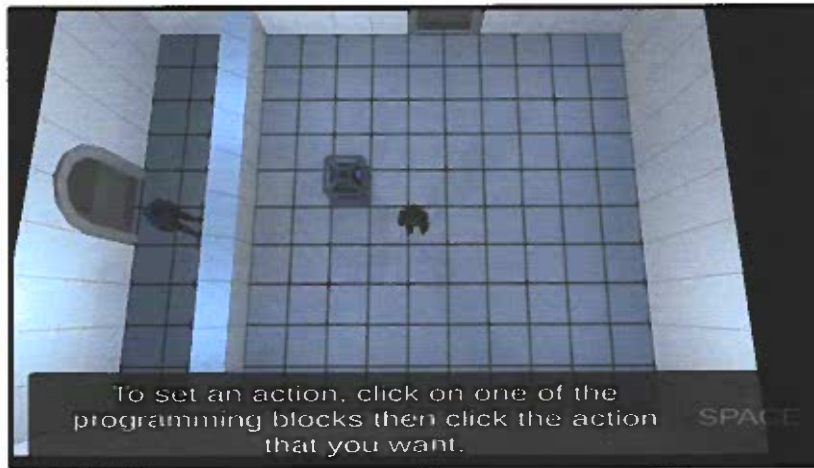
Game Screen Shots

Level Creation



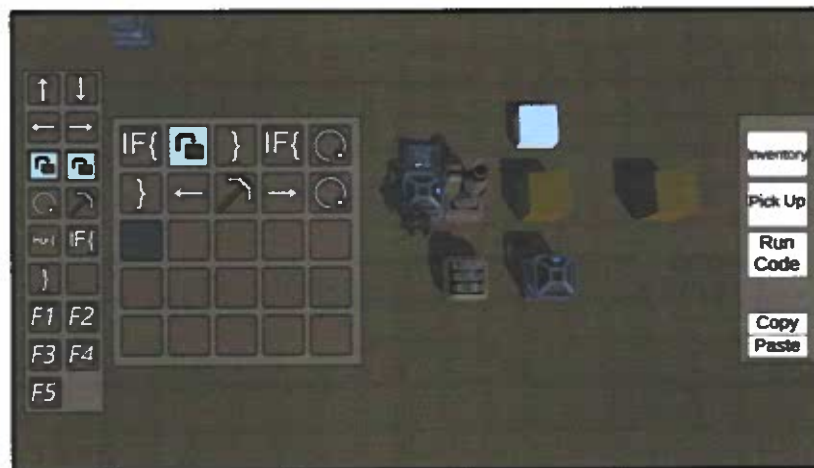
Game Screen Shots

Tutorial



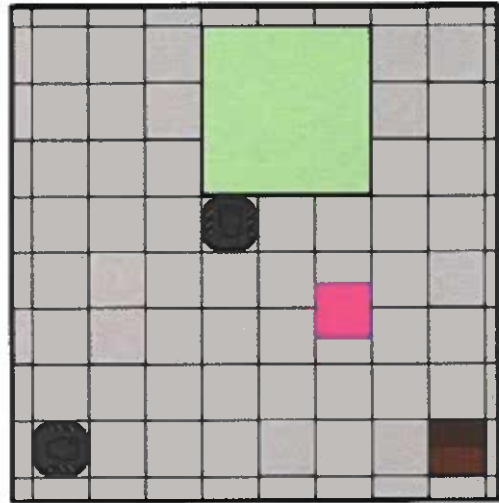
Game Screen Shots

In Game



■ Research / Design

- Prototype Design
 - Game Maker Studio
- References
 - YouTube
 - Discord



■ Software Used

- Implementation
 - Unity
 - Visual Studio (C#)
- Project Management
 - ClickUp

■ Project Requirements

Requirement Number/Type	3 Functional
Description	Can open player inventory GUI
Rationale	Player must be able to access the players inventory/crafting screen
Fit Criteria	During Game play the player GUI must open when the inventory button is pressed
Dependence	N/A
History	Created: 3/18/2019

■ Project Requirements

- 1) Ability to move player with WASD keys
- 2) Player character must not collide with other objects
- 3) Can open player inventory GUI
- 4) Can open Storage GUI
- 5) Can open Assembler GUI
- 6) Can Open Refinery GUI
- 7) Can open robot GUI
- 8) Can place all placeable objects into the world
- 9) Ability to Create a new game
- 10) Ability to Save the current game
- 11) Ability to load a previously saved world
- 12) Ability for the player to craft an item
- 13) Ability for the Assembler to craft an item
- 14) Ability for the Refinery to craft an item
- 15) Ability for the player to transfer items from the player's inventory to other inventories
- 16) Ability for the player to transfer items from other inventories to the players' inventory
- 17) Ability to program a Robot
- 18) Ability to Program the robot with every programming block
- 19) Robot must not collide with other objects
- 20) Player must be able to get every item
- 21) Every Item needs a model and / or a 2D sprite to visualize it
- 22) Every item that has a model must have their animation working

■ Test Plan

Play Testing

- 1) On a scale of 1-10 , How would you rate your overall experience while playing?
- 2) On a scale of 1-10, How easy was it to pick up on the controls?
- 3) On a scale of 1-10, How easy was it to learn how to program the robots?
- 4) Do you have any mechanics that you would like to see added?
- 5) Do you have any comments or suggestions about any aspect of the game?

■ Test Plan

Question 1:

On a scale of 1-10 , How would you rate your overall experience while playing?

Average: 7.75

■ Test Plan

Question 2:

On a scale of 1-10, How easy was it to pick up on the controls?

Average: 8.75

■ Test Plan

Question 3:

On a scale of 1-10, How easy was it to learn how to program the robots?

Average: 8.75

■ Test Plan

Question 4:

Do you have any mechanics that you would like to see added?

- Fighting System
- Build Walls
- Weather/Environmental Elements
- Hand Mining
- Variable Zoom
- Rotate Buildings
- Quick Access buttons to Crafting

■ Test Plan

Question 5:

Do you have any comments or suggestions about any aspect of the game?

- Less words at a time during tutorial
- Use visuals during tutorial
- Make the small world smaller
- Better introduction to controls for people who do not play PC